

Limbae Formale si Translatoare

Laborator 2

Februarie 2021

1 Scopul Lucrarii

1. Actiuni specifice LEX
2. Intelegerea notiunilor legate de dependentele de context stanga, Conditii de start
3. Reguli ambigue - continuare din lucrarea 1
4. Exemplu de analizor sintactic care foloseste yylex - procesarea unui fisier de configurare pentru o baza de date

2 Notiuni Teoretice

2.1 Actiuni specifice LEX

Ocazional, o actiune LEX poate decide ca o regula nu a recunoscut sirul de caractere corect. Pentru aceasta situatie sunt furnizate doua rutine, `yylex` si `yymore`, discutate mai in detaliu in Laboratorul 2.

- **`yymore()`**, ce poate fi apelata pentru a indica faptul ca urmatoarea expresie de intrare recunoscuta trebuie sa fie adaugata la sfirsitul ultimului sir de caractere recunoscut (in mod normal, urmatorul sir de intrare suprascrie sirul curent din `yytext`)
- **`yylex()`**, care indica faptul ca nu toate caracterele potrivite de expresia curenta sunt dorite acum. Argumentul `n` indica numarul de caractere de retinut din `yytext`, restul fiind returnate la intrare.

Acest lucru ofera acelasi gen de "privire inainte" ca cel oferit de operatorul `/`, dar intr-o forma diferita. De exemplu, sa consideram un limbaj ce defineste un sir ca fiind un set de caractere intre ghilimele, si cere ca aparitia unui caracter " (ghilimele) intr-un sir sa fie precedata de `\`. Se poate scrie si in felul urmator:

```
\["^"]*    {
            if(yytext[yytextlen-1]=='\\')
                yymore();
            else    <procesarea dorita de utilizator>
        }
```

Atunci cand va intalni un sir ca "abc\def", regula de mai sus va potrivi primele 5 caractere "abc" dupa care apelul la yymore() va face sa fie adaugata cealalta parte a sirului, adica "def". De notat ca semnul ghilimele final, care termina sirul trebuie sa fie tratat ("ciugulit") in codul <procesarea dorita de utilizator>.

Functia yyless() poate fi folosita pentru a reprocesa text in diferite circumstante. Considerand problema ce apare in limbajul C de a distinge ambiguitatea lui "=a", sa presupunem ca aparitia se doreste a fi tratata ca "= a", si sa se tipareasca si un mesaj. O rezolvare poate fi:

```
--[a-zA-Z]      {
                    printf("Operator (=-) ambiguu\n");
                    yyless(yyleng-1);
                    ... actiune pentru -= ...
                }
```

Solutia de mai sus tipareste mesajul, returneaza litera de dupa operator in fluxul de intrare, si trateaza operatorul drept "=". Alternativ, poate fi de dorit sa se trateze aparitia "=a" ca "= a". Pentru a realiza acest lucru se returneaza semnul minus si litera de la intrare:

```
--[a-zA-Z]      {
                    printf("Operator (=-) ambiguu\n");
                    yyless(yyleng-2);
                    ... actiune pentru = ...
                }
```

De notat ca expresiile pentru cele doua cazuri pot fi scrise mai usor astfel: $= -/[A - Za - z]$, in primul caz, si $= / - [A - Za - z]$, in al doilea caz, rezolvare in care nu trebuie efectuata nici o revenire in actiunea regulii. Nu e necesar sa fie recunoscut intregul identificator pentru a observa ambiguitatea. Oricum, posibilitatea "=3", face ca: $= -/[^\t\n]$, sa fie o regula mai buna.

2.2 Dependente de context stanga

Uneori, e de dorit sa avem mai multe seturi de reguli lexicale aplicabile intrarii la diferite momente. De exemplu, un preprocesor dintr-un compilator, ar putea distinge instructiunile preprocesor, pentru a le analiza altfel decit instructiunile ordinare. Acest caz necesita sensibilitate la context precedent, si poate fi tratat in mai multe moduri. Operatorul ^, de exemplu, e un operator de context precedent si recunoaste contextul stanga imediat precedent, asa cum \$ recunoaste contextul dreapta imediat urmator. Contextul stanga adiacent, poate fi extins pentru a produce o facilitate similara cu cea pentru contextul dreapta adiacent, dar e putin probabil ca acest lucru sa fie util, deoarece deseori contextul stanga relevant a aparut la un moment precedent, cum ar fi de exemplu inceputul liniei.

Exista trei cazuri de tratare a diferitelor medii (contexte sau "environments"):

1. utilizarea simpla a flag-urilor, cand doar cateva reguli "schimba" dintr-un mediu intr-altul
2. utilizarea conditiilor de start asupra regulilor
3. posibilitatea ca mai multe analizoare lexicale sa ruleze impreuna

In fiecare caz, exista reguli ce recunosc necesitatea de a schimba mediul in care urmatorul text de intrare sa fie analizat, si necesitatea de a seta un parametru pentru a reflecta schimbarea. Acest parametru poate fi un flag testat explicit de codul din actiunea utilizator. Un asemenea flag reprezinta modalitatea cea mai simpla de a solutiona problema, deoarece nu e implicat LEX-ul. Uneori poate fi mai convenabil ca LEX-ul sa memoreze flag-urile ca si conditii initiale asupra regulilor. Orice regula

poate fi asociata cu o conditie de start, caz in care ea va fi recunoscuta numai cand LEX-ul este in respectiva conditie de start. Conditia de start curenta poate fi modificata in orice moment.

Daca seturile de reguli pentru diversele medii sunt foarte diferite, pentru a obtine claritate e bine sa se scrie mai multe analizoare lexicale distincte, asigurandu-se comutarea de la unul la altul dupa necesitati.

Sa consideram problema copierii intrarii la iesire, cu modificarea cuvantului "magic" in "first" in fiecare linie ce incepe cu litera a, in "second" in fiecare linie ce incepe cu litera b, si in "third" in fiecare linie ce incepe cu litera c. Problema e foarte simpla, deci putem folosi doar un flag:

```

        int  flag;

%%
^a      { flag='a' ; ECHO; }
^b      { flag='b' ; ECHO; }
^c      { flag='c' ; ECHO; }
\n      { flag= 0  ; ECHO; }
magic   {
        switch(flag) {
            case 'a': printf("first");break;
            case 'b': printf("second");break;
            case 'c': printf("third");break;
            default : ECHO; break;
        }
    }

```

Pentru a rezolva problema folosind conditii de start, fiecare conditie de start trebuie mentionata in sectiunea de definitii printr-o linie de forma: %start name1 name2 cuvantul start poate fi inlocuit cu s sau S. Conditiiile pot fi referite la inceputul unei reguli folosind parantezele unghiulare (<>). De exemplu: <name1>expression, e o regula recunoscuta doar atunci cand LEX-ul e in conditia de start name1. Activarea unei conditii de start, se face cu instructiunea actiune:

BEGIN name1;

care schimba conditia de start in name1. Revenirea la starea normala se face cu instructiunea:

BEGIN 0;

care reseteaza conditia initiala a automatului LEX.

O regula poate fi activa in mai multe conditii de start, deci: < name1, name2, name3 > , e un prefix legal. Orice regula care nu incepe cu operatorul prefix % e activa permanent. Problema de mai sus poate fi rezolvata si astfel:

```

%START  AA  BB  CC
%%
^a      { ECHO; BEGIN  AA; }
^b      { ECHO; BEGIN  BB; }
^c      { ECHO; BEGIN  CC; }
\n      { ECHO; BEGIN  0; }
<AA>magic   printf("first");
<BB>magic   printf("second");
<CC>magic   printf("third");

```

Logica e aceeaasi ca in solutia precedenta, acum insa e folosit LEX-ul si nu codul utilizatorului.

2.3 Reguli Ambigue

Reamintire din primul laborator In situatiile in care mai mult de o expresie se poate potrivi cu intrarea curenta, LEX-ul decide dupa urmatoarele criterii:

1. Alege cea mai lunga potrivire
2. Dintre regulile care se potrivesc, si au acelasi numar de caractere, alege regula ce apare prima in tabela de reguli

REJECT In mod normal, LEX-ul partitioneaza fluxul de intrare, fara sa caute toate potrivirile posibile ale fiecarei expresii. Acest lucru inseamna ca fiecare caracter e contorizat o singura data. Sa presupunem de exemplu ca se doreste contorizarea atat a aparitiilor lui `he` cit si a aparitiilor lui `she` din intrare. O posibilitate de a realiza acest lucru este:

```
she      s++;
he       h++;
\n       |
.        ;
```

unde ultimele doua reguli ignora orice altceva in afara de `he` si `she`. Sa ne amintim ca `.` nu include "newline". Pentru ca `she` include `he`, in mod normal LEX-ul nu va recunoaste instantierile lui `he` incluse in `she`, deoarece o data ce a trecut de o instantiere `she`, respectivele caractere sunt pierdute.

Uneori, utilizatorul doreste sa modifice acest comportament implicit. Actiunea **REJECT** inseamna "du-te si executa urmatoarea alternativa". In acest caz e executata regula care e urmatoarea alternativa posibila (dupa regula curenta). Pozitia pointerului de intrare e ajustata corespunzator. Sa presupunem ca utilizatorul doreste sa contorizeze si instantierile lui `he` incluse in `she`. Urmatoarele reguli reprezinta un mod de a realiza acest lucru:

```
she      {s++; REJECT;}
he       {h++; REJECT;}
\n       |
.        ;
```

In acest exemplu, desigur, utilizatorul poate observa ca `she` include `he`, dar nu si viceversa, si poate omite actiunea **REJECT** pentru `he`. Sunt cazuri insa, in care s-ar putea sa nu se stie "a priori" ce caractere sunt in ambele clase.

2.4 Definitii in sursa LEX

Uneori utilizatorul are nevoie de optiuni aditionale, cum ar fi definirea unor variabile in scopul utilizarii lor de catre programul propriu sau de catre LEX. Acestea pot sa apara fie in sectiunea de definitii fie in sectiunea de reguli.

Sa ne amintim ca LEX-ul transforma regulile intr-un program. Orice cod sursa, neinterceptat de LEX, e copiat in programul generat. Exista 3 clase de asemenea cazuri:

1. Orice linie ce nu e parte a unei reguli sau actiuni LEX si care incepe cu "blanc" sau "tab", e copiată in functia generata de LEX. Astfel de cod sursa, daca apare inaintea primului delimitator `%`, va fi extern oricarei functii din programul generat. cand codul urmeaza imediat primului `%`, el va fi pus in partea de declaratii a functiei generate de LEX. Acest cod sursa trebuie sa preceada prima regula LEX. Ca un efect lateral, liniile ce incep cu "blanc" sau "tab", si care contin un comentariu, sunt trecute programului generat. Acest lucru poate fi folosit pentru a include comentarii atat in sursa LEX, cit si in codul generat. Comentariile trebuie sa urmeze conventiile limbajului gazda.
2. Codul sursa inclus intre liniile ce contin doar `{` si `}` e copiat in programul generat. Delimitatorii sunt ignorati. Acest format permite introducerea de text, cum ar fi de exemplu

instrucțiunile preprocesor ce trebuie să înceapă în coloana 1, sau linii ce nu par fragmente de programe.

3. Codul sursă ce apare după al doilea delimitator %% este copiat la sfârșitul codului generat de LEX.

Definițiile pentru LEX trebuie date înainte de primul delimitator %. Orice linie din această secțiune, necontinută între %{ și %} , și începând în coloana 1, e presupusă a defini siruri de substituție LEX. Formatul unei asemenea linii este: *name translation*, iar efectul ei e ca sirul dat ca *translation*, e asociat cu numele *name*. Numele și *translation* trebuie să fie separate prin cel puțin un spațiu sau un "tab", iar numele trebuie să înceapă cu o literă. *Translation* poate fi apelată (expandată) utilizând {*name*} în locul unde se dorește să apară *translation*.

Secțiunea de definiții poate conține și alte comenzi, inclusiv selecția limbajului gazdă, o tabelă cu setul de caractere, o listă de condiții de start iar pentru programe sursă mari expansiuni ale tablourilor LEX-ului față de dimensiunea implicată.

3 Desfasurarea Lucrării

3.1 Exercițiu Reject

Exersați REJECT: determinați aparițiile lui "she" și ale lui "he" pornind de la fișierul she.l.

Exemplu:

she reads, he cooks, she is happy, he is sad

she: 2 apariții, he: 4 apariții

she.l:

```
D      he
      int      s, h;

%%
s{D}      s++;
{D}      h++;
\n      ;
.      ;
%%
yywrap(){
    printf("She: %i; he:%i", s, h);
    return 1;
}
```

3.2 Condiții de Start

Testați exemplul cu condiții de start.

3.3 Procesarea unui fișier de tip configurarea a unei baze de date

Mergeți în directorul db_ config și urmați pașii următori, obiectivul fiind procesarea unui fișier de configurare de forma test.in. În fișierele de procesat, se vor permite cuvintele cheie db_name, db_port, db_type și db_table_prefix. Valorile pentru *tip*, *nume* și *table_prefix* sunt siruri de caractere mici și mari și cifre, dar trebuie să înceapă cu o literă, în timp ce valoarea portului e sir de cifre. Între un

cuvant cheie si valoarea asociata e necesar :, cu sau fara spatii. Orice altceva nu e permis si trebuie semnalat ca eroare. Ordinea in care apar sectiunile pt tip, nume, prefix si port e oricare.

- Pasul 1: creati un fisier de tip header cu toate simbolurile pe care tokenizer-ul ar trebui sa le recunoasca, numit `myscanner.h`
- Pasul 2: creati fisierul de intrare pentru lex
 - (a) includeti `myscanner.h` in prima parte a fisierului cu `%{ %}`
 - (b) scrieti actiunile si sabloanele

- Pasul 3:

```
> lex myscanner.l
> gcc myscanner.c lex.yy.c -ll -o myscanner
> echo "db_name:" | ./myscanner
Ar trebui sa obtineti un mesaj ca ati identificat un db_name si :.
> echo "db nam_e:" | ./myscanner
> echo "db_name : nosql" | ./myscanner
> ./myscanner < test.in
```

- Pasul 4: Verificati ordinea corecta a tokenilor in fisiere precum *test.in* - vezi `myscanner2.c`

- Pasul 5:

```
gcc myscanner2.c lex.yy.c -ll -o myscanner
./myscanner < test.in
Ar trebui sa obtineti:
db_type is set to mysql
db_name is set to testdata
db_table_prefix is set to test_
db_port is set to 1092

./myscanner < test_unexpected.in
db_type is set to mysql
unexpected character
db_name is set to testdata2
db_table_prefix is set to test_
db_port is set to 1092

./myscanner < test_error.in
db_type is set to mysql
Syntax error in line 1, expected an IDENTIFIER, but found 23
```